

Knowledge Base + RAGAS Evaluation — Complete Guide

Boiler & Chimney Agentic RAG System

Written: Senior Developer → Junior Developer | Every concept from zero

What you have already built: MQTT simulators running InfluxDB storing sensor data Grafana dashboard live Fault detector running Agent orchestrator with 4 tools FastAPI chatbot endpoint

What this document covers:

1. Knowledge Base — what it is, all 12 documents with full content, how to build it, how to update it when it changes
 2. RAGAS Evaluation — what it is, how it works, full code, how to read the scores
-

TABLE OF CONTENTS

- PART 1 — KNOWLEDGE BASE CONCEPTS (read before coding)
 - 1.1 What is a knowledge base and why you need it
 - 1.2 What goes into it — the 3 categories
 - 1.3 How ChromaDB stores and finds documents (visual)
- PART 2 — ALL 12 KNOWLEDGE BASE DOCUMENTS (full content)
 - 2.1 Boiler fault guides (6 documents)
 - 2.2 Chimney fault guides (2 documents)
 - 2.3 Multi-sensor diagnostic guides (2 documents)
 - 2.4 Sensor interpretation guides (2 documents)
- PART 3 — INDEXER CODE (build the knowledge base)
 - 3.1 indexer.py — full code explained line by line
 - 3.2 How to run it
 - 3.3 How to verify it worked
- PART 4 — KNOWLEDGE BASE CHANGES (most important section)
 - 4.1 4 scenarios when knowledge base changes
 - 4.2 How to update safely without breaking anything
 - 4.3 Version strategy for production
- PART 5 — RAGAS EVALUATION CONCEPTS (read before coding)
 - 5.1 What RAGAS is and why you need it
 - 5.2 The 3 metrics explained with boiler examples
 - 5.3 What scores mean and how to act on them
- PART 6 — RAGAS EVALUATOR CODE (full implementation)
 - 6.1 evaluator.py — full code explained line by line
 - 6.2 How scores are logged to InfluxDB
 - 6.3 How to see scores in Grafana
- PART 7 — STARTUP SEQUENCE (everything in order)

PART 1 — KNOWLEDGE BASE CONCEPTS

1.1 What is a knowledge base and why you need it

Your fine-tuned Gemini model knows boiler domain knowledge from training. But training knowledge has two problems:

Problem 1 — It cannot be updated without retraining. If your client adds a new fault type to their boiler, you cannot update the model's knowledge without spending hours fine-tuning again.

Problem 2 — It cannot cite sources. When the model says "HIGH_PRESSURE is caused by a stuck relief valve", you cannot verify where that came from or if it is accurate.

The knowledge base solves both problems:

WITHOUT knowledge base:

User: "Why is CO high?"
Agent: Calls search_knowledge_base("high CO causes")
→ Returns NOTHING (no knowledge base)
Agent: Falls back to training memory
→ Generic answer, possibly wrong, cannot verify

WITH knowledge base:

User: "Why is CO high?"
Agent: Calls search_knowledge_base("high CO causes")
→ Returns your HIGH_CO guide from ChromaDB
→ "Caused by insufficient combustion air,
burner nozzle fouling, blocked air filter..."
Agent: Answers using THIS specific text
→ Accurate, verifiable, updatable without retraining

Think of the knowledge base as a **reference manual on a bookshelf**. The agent can always pick up the right book and read the exact relevant page.

1.2 What goes into it — the 3 categories

Your knowledge base has exactly 3 types of documents:

CATEGORY 1: FAULT GUIDES (8 documents)

One document per fault code.
Each document answers: What is this fault? Why does it happen?
How do you fix it? How do you prevent it?

Documents:

- HIGH_PRESSURE fault guide
- LOW_WATER_LEVEL fault guide
- HIGH_TEMPERATURE fault guide
- LOW_FUEL_FLOW fault guide
- ABNORMAL_AIRFLOW fault guide
- HIGH_CO fault guide

- BLOCKED_FLUE fault guide
- HIGH_FLUE_TEMP fault guide

CATEGORY 2: MULTI-SENSOR DIAGNOSTIC GUIDES (2 documents)

When TWO OR MORE sensors are out of range simultaneously, the diagnosis is different from each fault alone. These documents explain combined-fault scenarios.

Documents:

- HIGH_PRESSURE + LOW_WATER together
- HIGH_CO2 + LOW_O2 together

CATEGORY 3: SENSOR INTERPRETATION GUIDES (2 documents)

Not about faults – about understanding what each sensor means. How to read it, what trends mean, seasonal variations.

Documents:

- CO2 percentage interpretation guide
- Chimney draft pressure interpretation guide

Total: 12 documents. Each document is plain text — no special formatting, no JSON. The embedding model converts them to vectors automatically.

1.3 How ChromaDB stores and finds documents (visual)

INDEXING PHASE (run once):

Your document text		Embedding Model		ChromaDB disk
"HIGH_PRESSURE fault occurs when pressure exceeds 14 bar. Causes: faulty..."	→	all-MiniLM-L6-v2 converts 500 words to 384 numbers [0.21, -0.54, ...]	→	Stores 3 things: 1. Original text 2. Vector [384 nums] 3. Metadata (title)

QUERY PHASE (runs on every agent tool call):

Agent calls: `search_knowledge_base("why is CO high in chimney")`

↓
Embedding model converts query to vector:
[0.68, 0.31, -0.22, ...]

↓
ChromaDB compares query vector against ALL stored vectors:

Document	Similarity
HIGH_CO fault guide	0.94 ← returned ▢
BLOCKED_FLUE guide	0.71 ← returned ▢
HIGH_PRESSURE guide	0.23
Water level guide	0.18

```
C02 interpretation          0.67 ← returned □
... (all 12 compared)

Returns top 3 most similar
(original TEXT, not vectors)
    |
    ▼
Agent reads the text and answers
```

PART 2 — ALL 12 KNOWLEDGE BASE DOCUMENTS

Create `knowledge_base/boiler_guides.py` with this exact content:

```
"""
knowledge_base/boiler_guides.py

All 12 knowledge base documents for the boiler and chimney system.
These are indexed into ChromaDB and retrieved by the agent at query time.

HOW TO ADD A NEW DOCUMENT:
1. Add a new dict to KNOWLEDGE_DOCUMENTS following the same format
2. Give it a unique "id" (lowercase, underscores, no spaces)
3. Give it a descriptive "title"
4. Write the "content" as plain text (no markdown, no JSON)
5. Run: python knowledge_base/indexer.py --mode=add --id=your_new_id
   (or full re-index: python knowledge_base/indexer.py --mode=full)

HOW TO UPDATE AN EXISTING DOCUMENT:
1. Find the document by its "id" in this list
2. Edit the "content" text
3. Run: python knowledge_base/indexer.py --mode=update --id=the_document_id

HOW TO REMOVE A DOCUMENT:
1. Delete the dict from KNOWLEDGE_DOCUMENTS
2. Run: python knowledge_base/indexer.py --mode=full
   (full re-index is safest for deletions)
"""

KNOWLEDGE_DOCUMENTS = [

    # =====
    # CATEGORY 1A – BOILER FAULT GUIDES
    # =====

    {
        "id": "fault_high_pressure",
        "category": "boiler_fault",
        "title": "HIGH_PRESSURE Fault – Complete Guide",
        "content": """
HIGH_PRESSURE fault code is triggered when boiler steam pressure exceeds 14 bar.
This is a CRITICAL severity fault. Immediate action is required.
Normal operating range for boiler pressure is 10 to 14 bar.

WHAT IS HAPPENING:
The boiler is generating steam pressure faster than it is being consumed or
released.
Pressure will continue rising until the cause is removed or the pressure relief
valve opens.
```

Above 16 bar, mechanical failure risk increases significantly.
Above 18 bar, explosion risk becomes serious.

ROOT CAUSES (in order of likelihood):

- Cause 1: Pressure relief valve stuck closed or set too high.
The relief valve is the safety device that should automatically open when pressure exceeds the setpoint (usually 14.5 to 15 bar).
If it is stuck, pressure keeps rising with no automatic safety release.
- Cause 2: Downstream steam outlet valve closed or partially closed.
If the valve that lets steam leave the boiler is closed, the steam has nowhere to go. Pressure builds inside the vessel.
- Cause 3: Burner firing rate too high for current steam demand.
If steam demand has dropped (e.g. a downstream process stopped) but the burner is still firing at the same rate, the boiler makes more steam than is being used. Pressure rises.
- Cause 4: Scale buildup on heat exchanger surfaces.
Mineral scale insulates the heating surfaces. More heat stays in the combustion chamber instead of transferring to water.
This causes localised overheating which drives up pressure.
- Cause 5: Boiler overfed with water, water level too high.
Excess water reduces the steam space, reducing volume for pressure equalisation and causing rapid pressure rise.

STEP BY STEP IMMEDIATE ACTIONS:

- Step 1: Reduce burner to minimum firing rate immediately.
Do not shut off completely yet unless pressure exceeds 16 bar.
Gradual reduction prevents thermal shock.
- Step 2: Check downstream steam outlet valve is fully open.
Walk to the main steam header and verify isolation valves are open.
- Step 3: Manually test the pressure relief valve.
Lift the test lever briefly. Steam should escape freely.
If no steam comes out, the valve is stuck. This is an emergency.
- Step 4: If pressure reaches 16 bar, shut off burner completely.
- Step 5: If pressure reaches 18 bar, evacuate the boiler room.
This is a potential explosion risk situation.
- Step 6: Do not restart until root cause is identified and fixed.
- Step 7: Call a certified boiler engineer for inspection before restart.

PREVENTION MEASURES:

Test pressure relief valve monthly using the lift test lever.
Install a pressure trend alert at 12 bar to warn before reaching 14 bar.
Schedule annual boiler inspection including relief valve calibration.
Monitor downstream steam demand and adjust burner firing rate accordingly.
Perform annual chemical descaling to prevent scale-related pressure issues.

IMPORTANT SAFETY NOTE:

Never operate a boiler above 15 bar without engineering authorisation.
The Indian Boiler Regulations (IBR) require that all pressure-containing components are rated and certified for the maximum allowable working pressure.

""""

},

{

"id": "fault_low_water_level",
"category": "boiler_fault",
"title": "LOW_WATER_LEVEL Fault – Complete Guide",
"content": """"

LOW_WATER_LEVEL fault code is triggered when boiler drum water level drops below 40 percent. This is a CRITICAL severity fault.
Normal drum water level range is 40 to 60 percent.

THIS IS THE MOST DANGEROUS BOILER CONDITION.

A boiler with insufficient water will overheat its tubes.
Overheated tubes warp, crack, and can rupture violently.

A tube rupture releases high-pressure steam explosively.
Most industrial boiler explosions in history trace to low water.

WHAT IS HAPPENING:

Water is leaving the boiler faster than it is being replaced.
The exposed heating surfaces (tubes, furnace walls) are receiving heat from combustion but have no water to absorb it.
Metal temperature rises rapidly. Above 450 degrees Celsius, carbon steel tubes lose structural integrity.

ROOT CAUSES (in order of likelihood):

Cause 1: Feed water pump failure.

The pump that continuously supplies water to the boiler has failed electrically (tripped breaker, burnt motor) or mechanically (worn impeller, cavitation, seized shaft).

Feed water flow rate drops to zero. Level falls rapidly.

Cause 2: Feed water control valve stuck closed.

The automatic valve controlling water supply has failed closed.

Pump is running but water cannot pass through.

Cause 3: Steam demand suddenly increased beyond feed capacity.

A large steam consumer came online unexpectedly.

Water is converted to steam faster than the pump can replace it.

Cause 4: Boiler tube or fitting leak.

Water is escaping through a crack or failed fitting.

Look for unexpected steam clouds or water on the boiler room floor.

Cause 5: Water level gauge or sensor failure.

The level reading may be incorrect.

The actual water level could be normal.

Always visually verify before acting on a sensor reading.

STEP BY STEP IMMEDIATE ACTIONS:

Step 1: SHUT OFF BURNER IMMEDIATELY. No exceptions.

This is mandatory. You cannot run a low-water boiler.

Step 2: Do NOT add cold water to a hot boiler.

If the boiler has been running hot without water for even a few minutes, the tubes are extremely hot. Cold water on hot dry tubes causes thermal shock and tube cracking.

This is called a cold water incident and causes more damage than the original low water condition.

Step 3: Verify the level reading with physical inspection.

Look at the sight glass directly. Is the reading accurate?

Step 4: Check feed water pump operation.

Is it running? Check the electrical panel for a tripped breaker.

Is suction valve open? Is discharge valve open?

Step 5: Check feed water control valve position.

Is it open? Try manual override if available.

Step 6: Inspect visible boiler surfaces for steam or water leaks.

Look for unusual moisture, steam clouds, or water on the floor.

Step 7: Let boiler cool to below 60 degrees Celsius before restoring water if the pump was off for more than 10 minutes.

Step 8: Call a certified boiler engineer before restarting.

A qualified engineer must inspect for tube damage before the boiler is returned to service.

PREVENTION MEASURES:

Inspect feed water pump weekly. Check impeller wear quarterly.

Test low water cutoff device monthly using the slow drain test.

Install duplicate water level sensors for redundancy.

Install automatic feed water control with low-level alarm at 45 percent.

Check drum level manually every 4 hours during operation.

Maintain a boiler log with level readings at each shift change.

""""

},

```
{
  "id": "fault_high_temperature",
  "category": "boiler_fault",
  "title": "HIGH_TEMPERATURE Fault – Complete Guide",
  "content": ""
```

HIGH_TEMPERATURE fault code is triggered when boiler water or steam temperature exceeds the normal maximum of 100 degrees Celsius above the design specification. This is a WARNING severity fault.
Normal boiler water temperature range is 60 to 100 degrees Celsius.

WHAT IS HAPPENING:

The boiler water is absorbing more heat than it should at current operating conditions. This may indicate a heat transfer problem, an over-firing condition, or a water circulation issue.
If left uncorrected it can progress to a CRITICAL condition.

ROOT CAUSES:

Cause 1: Scale or sludge deposits on heating tube surfaces.

Mineral scale is a poor conductor of heat.

A scale layer of just 1 millimetre on the tube inner surface reduces heat transfer efficiency by approximately 10 percent.

The water does not absorb heat efficiently, so temperature rises.

This is the most common long-term cause of high temperature.

Cause 2: Burner over-firing.

Burner is delivering more heat than the boiler design allows.

Check burner settings and fuel pressure.

Cause 3: Low water circulation rate.

In a circulation boiler, a pump moves water through the heating tubes. If this pump slows or fails, water stays in the hot zone longer than designed and picks up more heat.

Cause 4: Incorrect fuel-to-air ratio running rich.

Too much fuel relative to air produces a hotter flame.

The combustion temperature is higher, pushing more heat into the water.

Cause 5: Blocked flue gases inside boiler passages.

If soot or debris blocks the internal gas passages, combustion products spend more time inside the boiler transferring heat.

RELATIONSHIP WITH FLUE TEMPERATURE:

If boiler water temperature is high AND chimney flue temperature is also high simultaneously, scale buildup is almost certainly the cause. Heat cannot pass through scale to the water, so both the combustion side (high flue temp) and the water side (high water temp) run hot. This dual-high pattern is the classic scale signature.

CONTROL ACTIONS:

Reduce burner firing rate by 20 percent increments.

Check water circulation pump speed and operation if fitted.

Measure flue gas CO2 percentage. If above 14 percent, mixture is too rich. Open combustion air damper.

If scale buildup is suspected: schedule chemical descaling treatment.

Do not delay. Scale buildup worsens over time and will eventually cause HIGH_PRESSURE and efficiency loss as well.

PREVENTION MEASURES:

Annual chemical descaling treatment.

Monthly water quality testing. Hardness should be below 50 ppm.

Quarterly combustion analysis to verify correct fuel-air ratio.

Trend monitoring of flue temperature.

A flue temperature rising by 5 degrees per week indicates active scale formation. Schedule descaling before it worsens.

""

```
},
```

```
{
  "id": "fault_low_fuel_flow",
  "category": "boiler_fault",
  "title": "LOW_FUEL_FLOW Fault – Complete Guide",
  "content": ""
}
```

LOW_FUEL_FLOW fault code is triggered when fuel flow rate drops below 5 litres per minute. Normal range is 5 to 20 litres per minute. This is a WARNING severity fault.

WHAT IS HAPPENING:

The burner is receiving less fuel than required for the current heat demand. Steam production will drop. If severe enough, the burner may extinguish. Pressure and temperature will fall.

ROOT CAUSES:

Cause 1: Fuel supply pressure drop.

The fuel supply line pressure has dropped. Could be supply network pressure reduction, or a valve partially closing.

Cause 2: Fuel filter blocked or heavily fouled.

Fuel filters collect debris from the supply line.

A blocked filter restricts fuel flow to the burner.

Cause 3: Fuel control valve stuck or failing.

The valve that regulates fuel delivery to the burner has stuck in a partially closed position.

Cause 4: Burner nozzle partially blocked.

The nozzle that atomises fuel has carbon deposits narrowing the orifice, restricting flow.

Cause 5: Fuel pump wear or failure.

The pump delivering fuel from the storage tank to the burner has reduced output due to wear.

CONTROL ACTIONS:

Check fuel supply line pressure at the meter or supply connection.

Inspect and clean or replace the fuel filter.

Verify fuel control valve is in the fully open position.

Inspect burner nozzle for carbon deposits. Clean with solvent.

Check fuel pump delivery pressure against specification.

PREVENTION:

Replace fuel filter every 3 months or after heavy use periods.

Annual burner service including nozzle inspection and cleaning.

Install fuel pressure gauge with low-pressure alarm.

""

```
},
```

```
{
  "id": "fault_abnormal_airflow",
  "category": "boiler_fault",
  "title": "ABNORMAL_AIRFLOW Fault – Complete Guide",
  "content": ""
}
```

ABNORMAL_AIRFLOW fault code is triggered when combustion air flow falls below 100 cubic metres per hour or exceeds 300 cubic metres per hour. Normal range is 100 to 300 cubic metres per hour.

T00 LOW (below 100 cubic metres per hour):

Insufficient combustion air reaching the burner.

Result: incomplete combustion, high CO, high CO₂, black smoke, inefficient operation, potential burner instability.

Causes: blocked combustion air filter, air damper stuck closed, blocked air intake duct, burner fan failure or low speed.

T00 HIGH (above 300 cubic metres per hour):

Excess air diluting combustion gases unnecessarily.

Result: flame cooling, reduced efficiency, high fuel consumption,

excessive heat loss through chimney, low CO₂ in flue gas.
Causes: air damper stuck open, combustion analysis not performed,
leaking boiler casing allowing cold air infiltration.

COMBUSTION AIR RELATIONSHIP WITH FLUE GAS:

Correct air supply produces: CO₂ 10-13 percent, O₂ 3-6 percent, CO below 30 ppm.
Low air produces: CO₂ above 14 percent, O₂ below 3 percent, CO above 50 ppm.
Excess air produces: CO₂ below 8 percent, O₂ above 8 percent, CO near zero.

CONTROL ACTIONS:

Measure current O₂ and CO₂ percentages in flue gas.
Adjust combustion air damper in small increments.
After each adjustment, wait 2 minutes for readings to stabilise.
Target O₂ of 4 to 6 percent with CO₂ of 10 to 12 percent.
Check air filter and replace if blocked.
Verify fan motor speed is correct.

PREVENTION:

Replace combustion air filter every 3 months.
Annual combustion analysis and burner tuning by certified engineer.
Quarterly flue gas spot checks using portable analyser.

},

=====
CATEGORY 1B – CHIMNEY FAULT GUIDES
=====

{
 "id": "fault_high_co",
 "category": "chimney_fault",
 "title": "HIGH_CO Fault – Complete Guide",
 "content": ""

HIGH_CO fault code is triggered when carbon monoxide concentration
in chimney flue gas exceeds 50 ppm (parts per million).
This is a CRITICAL severity fault. Personnel safety risk.
Normal CO range in flue gas is 0 to 50 ppm.

WHAT CARBON MONOXIDE IS:

Carbon monoxide (CO) is produced when fuel does not burn completely.
In complete combustion: fuel + enough air = CO₂ + water vapour.
In incomplete combustion: fuel + insufficient air = CO + water vapour.
CO is colourless and has no smell. You cannot detect it without instruments.
CO is poisonous. It binds to haemoglobin in blood, preventing oxygen transport.

PERSONNEL SAFETY THRESHOLDS (mandatory knowledge):

0 to 50 ppm: Normal operation range. No health risk.
50 to 200 ppm: WARNING. Investigate and increase ventilation immediately.
 At 200 ppm: symptoms appear after 2 to 3 hours (headache, dizziness).
200 to 400 ppm: Evacuate boiler room. Life-threatening within 3 hours.
Above 800 ppm: Life-threatening within 45 minutes. Emergency evacuation.
Above 1600 ppm: Fatal within 1 hour.

INSTALL A CO DETECTOR ALARM IN EVERY BOILER ROOM.

This is a legal requirement under Indian factory safety regulations.

ROOT CAUSES:

Cause 1: Insufficient combustion air supply.
 Not enough oxygen reaching the burner for complete combustion.
 This is the most common cause.
 Check: O₂ below 3 percent and CO₂ above 14 percent together
 confirm this diagnosis.
Cause 2: Burner nozzle fouling with carbon deposits.
 The nozzle cannot atomise fuel correctly.

Fuel droplets are too large to burn completely.
Check: CO is high but O2 is normal (enough air, but poor mixing).
Cause 3: Cracked heat exchanger or furnace wall.
Combustion gases are bypassing the normal flow path.
They may be mixing with room air instead of going up the chimney.
Check: CO in boiler room air, not just in the flue gas.
Cause 4: Blocked combustion air intake filter.
Filter is clogged with dust. Air supply is restricted.
Check: Air flow rate is low, filter is visibly dirty.
Cause 5: Fuel pressure too low causing poor atomisation.
Insufficient fuel pressure means the nozzle cannot break
fuel into fine enough droplets for complete combustion.

STEP BY STEP CONTROL ACTIONS:

Step 1: If CO exceeds 200 ppm – ventilate the boiler room immediately.
Open all doors and windows. Turn on ventilation fans.
Step 2: Open combustion air damper by 10 percent increments.
Step 3: Measure O2 in flue gas after each damper adjustment.
O2 should rise as you add more air. Target: 4 to 6 percent O2.
Step 4: If O2 rises but CO stays high, burner nozzle fouling is likely.
Shut down burner and inspect nozzle. Clean or replace.
Step 5: Check and replace combustion air filter if blocked.
Step 6: If CO exceeds 400 ppm – shut down boiler immediately.
Evacuate all personnel from boiler room.
Step 7: Inspect heat exchanger for cracks before restarting.

PREVENTION:

Install CO detector with alarm in boiler room. Test monthly.
Annual burner service including nozzle cleaning and replacement.
Replace combustion air filter every 3 months.
Quarterly flue gas CO measurement using portable analyser.
Maintain boiler room ventilation. Minimum 10 air changes per hour.
""

},

{

"id": "fault_blocked_flue",
"category": "chimney_fault",
"title": "BLOCKED_FLUE Fault – Complete Guide",
"content": ""

BLOCKED_FLUE fault code is triggered when chimney draft pressure
becomes less negative than minus 2 Pascals.
Normal draft pressure range is minus 2 to minus 5 Pascals.
This is a CRITICAL severity fault.

UNDERSTANDING DRAFT PRESSURE:

Draft is the negative pressure (suction) inside the chimney.
It is created by the difference in density between hot flue gas
inside the chimney and cooler air outside.
Hot gases are less dense than cool air. They rise. This rising
creates suction at the bottom, pulling combustion air through
the burner and drawing flue gases up and out.
Draft is always expressed as a negative number during operation.
More negative means stronger suction. Less negative means weaker.
Zero means no draft. Positive means reverse flow (back pressure).

WHAT IS HAPPENING:

The chimney cannot pull flue gases upward effectively.
Combustion gases may back-flow into the boiler room.
This creates CO accumulation risk in the boiler room.

ROOT CAUSES:

Cause 1: Soot and ash accumulation inside chimney flue.
Over months of operation, combustion residues build up on

the inner chimney wall. This narrows the effective diameter, increasing flow resistance and reducing draft.

Annual chimney sweeping prevents this.

Cause 2: Physical obstruction at chimney outlet.

Bird nests are extremely common in industrial chimneys.

Debris, leaves, or dead birds can partially block the outlet.

A visual inspection from ground level often reveals this.

Cause 3: Chimney liner collapse or structural damage.

The internal liner of the chimney has fractured or shifted.

Requires camera inspection to diagnose.

Cause 4: Downdraught from wind.

Wind direction and speed can temporarily push air back down the chimney. This is especially common in urban areas with nearby tall buildings creating complex wind patterns.

A chimney cowl prevents most downdraught conditions.

Cause 5: Building negative pressure.

The building HVAC system is extracting more air from the building than it is supplying. This creates negative pressure inside the building, which can reverse chimney flow.

Seen in tightly sealed modern buildings.

CONTROL ACTIONS:

Shut down boiler. Do not operate with a blocked flue.

The risk of CO accumulation in the boiler room is too high.

With boiler off, visually inspect chimney outlet from ground level.

Look for obvious obstructions (nests, debris, cowl damage).

Use a draft gauge at multiple heights to locate blockage zone.

Schedule professional chimney sweep for soot removal.

For suspected structural collapse: arrange camera CCTV inspection before scheduling a sweep (sweeping a collapsed liner can displace debris and worsen the blockage).

PREVENTION:

Annual chimney sweep by qualified sweep contractor.

Install a chimney cowl (terminal) to prevent bird nesting and reduce downdraught.

Monthly draft pressure monitoring. Trend below minus 2 Pa

for more than 3 consecutive readings = schedule inspection soon.

Inspect chimney exterior and outlet after severe storms.

},

{

"id": "fault_high_flue_temp",

"category": "chimney_fault",

"title": "HIGH_FLUE_TEMPERATURE Fault – Complete Guide",

"content": ""

HIGH_FLUE_TEMPERATURE fault code is triggered when chimney outlet temperature exceeds 250 degrees Celsius.

Normal flue temperature range is 150 to 250 degrees Celsius.

This is a WARNING severity fault.

WHAT IS HAPPENING:

Combustion gases are exiting the chimney hotter than they should.

This means the boiler heat exchanger is not transferring as much heat from the flue gases to the water as it is designed to.

Hot flue gases represent wasted energy leaving through the chimney.

EFFICIENCY IMPACT (critical for client cost awareness):

Every 10 degrees Celsius above optimal flue temperature equals approximately 1 percent reduction in boiler thermal efficiency.

Example: Optimal flue temperature is 200 degrees Celsius.

Measured flue temperature is 260 degrees Celsius.

Difference is 60 degrees Celsius.

Efficiency loss: approximately 6 percent.
For a boiler consuming 50,000 litres of fuel per year:
6 percent waste equals 3,000 litres of fuel wasted annually.
At current fuel prices, this is a significant operating cost.
The client should understand this fault directly costs them money.

ROOT CAUSES:

Cause 1: Scale buildup on boiler heat exchanger tubes.

This is by far the most common cause.

Mineral deposits from hard water accumulate on tube surfaces.

Scale is a thermal insulator. Heat cannot transfer through it.

Flue gases stay hot because the water cannot absorb their heat.

Signature: Flue temperature rises gradually over weeks or months.

Cause 2: Excessive excess air in combustion.

Too much cold combustion air is passing through the boiler.

This cool air dilutes the hot flue gases and passes straight through without useful heat transfer.

O₂ above 8 percent confirms excess air condition.

Cause 3: Missing or damaged flue gas baffles inside boiler.

Baffles force flue gases to take a longer path through the heat exchanger, increasing contact time and heat transfer.

If baffles are missing or broken, gases take a shortcut.

Cause 4: Boiler operating significantly above design capacity.

Higher heat input than designed means gases move faster through the heat exchanger, reducing heat transfer time.

Cause 5: Short-cycling operation.

Boiler starts, quickly reaches temperature, shuts off.

During start-up, the heat exchanger is cold and absorbs heat rapidly. But during short on-periods, the exchanger never reaches steady-state efficient operation.

TREND ANALYSIS (how to detect scale buildup early):

Monitor flue temperature daily.

If flue temperature rises more than 10 degrees per week with no change in burner settings or fuel type, scale buildup is almost certainly occurring.

Schedule descaling within 2 to 4 weeks.

Do not wait for the fault threshold to be reached.

Early intervention costs less than emergency descaling.

CONTROL ACTIONS:

Check O₂ percentage. If above 8 percent, reduce air supply.

Close combustion air damper by 5 percent increments.

Recheck O₂ after each adjustment. Target 4 to 6 percent.

Inspect flue baffles during next planned maintenance outage.

Replace any missing or cracked baffles.

Schedule chemical descaling if scale is suspected.

Review boiler load profile. Is it being asked to short-cycle?

Consider installing a thermal buffer tank to smooth demand.

PREVENTION:

Monthly flue temperature recording and trend monitoring.

Annual heat exchanger inspection and tube cleaning.

Quarterly water hardness testing. Treat if above 50 ppm hardness.

Quarterly combustion analysis to verify correct air-fuel ratio.

""
},

CATEGORY 2 – MULTI-SENSOR DIAGNOSTIC GUIDES
#

{
"id": "diag_pressure_plus_low_water",

```
        "category": "multi_sensor_diagnostic",
        "title": "Combined Diagnosis: HIGH_PRESSURE + LOW_WATER_LEVEL
Simultaneously",
        "content": ""
When HIGH_PRESSURE fault and LOW_WATER_LEVEL fault occur at the same time,
this is a DOUBLE CRITICAL emergency situation requiring immediate boiler
shutdown.
```

DO NOT TRY TO FIX ONE FAULT WHILE THE OTHER PERSISTS.
The combination is more dangerous than either fault alone.

WHAT IS HAPPENING:

The feed water system has failed while the burner continues to fire.
Water in the boiler is being converted to steam faster than it is
being replaced by feed water.
As water level drops, steam space increases, causing pressure to rise.
Simultaneously, exposed heating surfaces are overheating due to lack of water.
This is a mechanical and safety emergency.

MOST LIKELY ROOT CAUSE:

Feed water pump has failed mechanically or electrically.
The pump stopped delivering water. The burner continued firing.
Over several minutes, water level has dropped and pressure has risen.

ALTERNATIVE CAUSE:

A boiler tube has cracked or failed.
Water is escaping through the breach.
This is less common but more structurally serious.
Look for unexpected steam or water emerging from the boiler casing.

IMMEDIATE RESPONSE (follow in exact order):

Step 1: SHUT OFF BURNER IMMEDIATELY.

This is the single most important action. Remove the heat source.

Step 2: DO NOT OPEN THE SAFETY RELIEF VALVE MANUALLY.

Steam at high pressure escaping in a confined space is extremely
dangerous to personnel. Let the pressure drop naturally.

Step 3: DO NOT ADD COLD WATER TO A HOT LOW-WATER BOILER.

This causes thermal shock and can crack tubes.

Wait until boiler cools to below 60 degrees Celsius.

Step 4: Isolate the boiler from the steam distribution system.

Close the main steam outlet valve.

Step 5: DO NOT RESTART THE BOILER.

This boiler must not be restarted until inspected by a certified
boiler engineer. Tube damage may have occurred. Operating a boiler
with damaged tubes can cause catastrophic failure.

Step 6: Contact a certified boiler inspector.

Under Indian Boiler Regulations, a certified inspector must
examine the boiler before it can legally return to service
after a low water incident.

ROOT CAUSE INVESTIGATION AFTER SHUTDOWN:

Check feed water pump electrical supply. Check breaker in panel.

Check pump mechanical state. Attempt to turn shaft by hand.

Check suction and discharge valves. Are they open?

Check feed water control valve. Is it open or stuck?

Review monitoring history. How long was the pump off before alarm?

Inspect boiler exterior for leaks, cracks, or distorted surfaces.

```
""
```

```
},
```

```
{
```

```
    "id": "diag_high_co2_low_o2",
```

```
    "category": "multi_sensor_diagnostic",
```

```
    "title": "Combined Diagnosis: HIGH_CO2 + LOW_O2 in Flue Gas",
```

"content": ""

When CO2 percentage in flue gas exceeds 14 percent AND oxygen percentage is below 3 percent simultaneously, this confirms a rich combustion mixture. Too much fuel relative to combustion air. Incomplete combustion is occurring. CO will also be elevated in this situation.

UNDERSTANDING THE RELATIONSHIP:

CO2 and O2 in flue gas always move in opposite directions.

When air supply increases: O2 rises, CO2 falls, CO falls.

When air supply decreases: O2 falls, CO2 rises, CO rises.

CO2 above 14 percent with O2 below 3 percent is an unambiguous signal that combustion air is insufficient for the fuel being burned.

WHY THIS MATTERS:

Incomplete combustion wastes fuel. Carbon that should become CO2 instead becomes CO, which goes up the chimney as unburned energy.

CO is toxic to personnel in the boiler room.

A rich mixture can also cause sooting of heat exchanger surfaces and burner components, requiring more frequent maintenance.

DIAGNOSIS CERTAINTY:

CO2 high + O2 low together means insufficient air. This is definitive.

You do not need to check other causes when both readings confirm this.

STEP BY STEP CONTROL:

Step 1: Open combustion air damper by 10 percent.

Make one adjustment at a time. Do not jump to large changes.

Step 2: Wait 2 to 3 minutes for readings to stabilise.

Flue gas composition lags behind damper changes by 1 to 3 minutes.

Step 3: Measure O2 and CO2 again.

O2 should have risen. CO2 should have fallen.

CO should also be falling if air was the root cause.

Step 4: Continue adjusting damper in 5 percent increments

until O2 reaches 4 to 6 percent and CO2 reaches 10 to 12 percent.

Step 5: If O2 rises but CO remains high after air correction:

The problem is now burner nozzle fouling, not air supply.

Shut down and inspect nozzle. Clean or replace as needed.

Step 6: Check combustion air filter condition.

If filter is visibly dirty, replace it. A dirty filter causes gradual drift toward rich combustion over weeks.

TARGET FLUE GAS COMPOSITION:

O2: 4 to 6 percent (optimal for natural gas boilers)

CO2: 10 to 12 percent

CO: below 30 ppm

Flue temperature: 150 to 220 degrees Celsius

PREVENTION:

Quarterly flue gas analysis and burner adjustment.

Air filter replacement every 3 months.

Annual burner service by qualified engineer.

""

},

#

CATEGORY 3 – SENSOR INTERPRETATION GUIDES

#

{

"id": "guide_co2_interpretation",

"category": "sensor_interpretation",

"title": "How to Interpret CO2 Percentage in Chimney Flue Gas",

"content": ""

CO2 percentage in chimney flue gas is the primary indicator of

combustion efficiency for a boiler. Understanding what CO₂ readings mean allows early detection of combustion problems.

WHAT CO₂ IN FLUE GAS REPRESENTS:

CO₂ is the product of complete combustion of carbon in fuel.

Higher CO₂ means more carbon has been fully burned per unit of air passing through.

Lower CO₂ means either excess air is diluting the gases or incomplete combustion is occurring.

READING INTERPRETATION GUIDE:

CO₂ below 8 percent:

Excessive combustion air. Too much cold air passing through.

Fuel is being burned but most of the flue gas is diluted with excess air that served no combustion purpose.

Result: Efficiency loss, high fuel consumption, unnecessarily high flue temperature.

Action: Reduce air supply. Close combustion air damper.

CO₂ between 8 and 12 percent:

Good combustion. Adequate but not excessive air supply.

Most boilers operate well in this range.

No immediate action needed.

CO₂ between 12 and 14 percent:

Excellent combustion efficiency. Near-stoichiometric conditions.

The boiler is burning fuel as efficiently as practical.

This is the ideal target range for most natural gas boilers.

No action needed.

CO₂ above 14 percent:

Insufficient combustion air. Rich mixture.

Carbon cannot find enough oxygen for complete combustion.

CO will also be elevated. Personnel safety risk.

Action: Increase air supply immediately.

RELATIONSHIP WITH O₂:

CO₂ and O₂ always move in opposite directions.

As you add more air: O₂ rises, CO₂ falls.

As you reduce air: O₂ falls, CO₂ rises.

Use both readings together for precise diagnosis.

If CO₂ high AND O₂ low: definitely insufficient air.

If CO₂ low AND O₂ high: definitely excessive air.

TREND ANALYSIS OVER TIME:

CO₂ rising gradually over weeks without changing settings:

Likely cause: combustion air filter becoming blocked.

Less air entering means proportionally more CO₂.

Check and replace filter.

CO₂ dropping gradually over weeks without changing settings:

Likely cause: air leaking into flue gas path from casing cracks.

Cool air dilutes flue gases. CO₂ percentage drops.

Inspect boiler casing and flue connections for gaps.

CO₂ fluctuating widely and rapidly:

Indicates combustion instability.

Could be burner cycling, fuel pressure fluctuations, or air damper mechanical problem.

Investigate burner control system.

SEASONAL VARIATION:

In very cold weather, incoming combustion air is denser.

More oxygen atoms per cubic metre of air.

The same air flow setting will produce slightly richer mixture.
CO2 may rise by 0.5 to 1 percent in winter. This is normal.
Adjust damper slightly in winter for precise tuning.

```
    },  
  
    {  
      "id": "guide_draft_pressure",  
      "category": "sensor_interpretation",  
      "title": "How to Interpret Chimney Draft Pressure",  
      "content": ""
```

Draft pressure is the negative pressure (suction) measured inside the chimney flue. It is the driving force that pulls combustion air through the burner and expels flue gases safely upward. Draft is always a negative value during normal boiler operation.

UNDERSTANDING NEGATIVE PRESSURE:

Negative pressure means the pressure inside the chimney is lower than atmospheric pressure outside.
This lower pressure sucks air in at the burner end and pushes flue gases up and out at the chimney top.
More negative = stronger suction = better draft.
Less negative = weaker suction = poorer draft.
Zero or positive = no draft or reverse flow = dangerous condition.

NORMAL OPERATING RANGE:

The normal draft pressure range is minus 2 to minus 5 Pascals. This provides sufficient suction for combustion air flow without excessive heat loss from over-drafting.

READING INTERPRETATION:

Draft between 0 and minus 2 Pascals (insufficient draft):

The chimney cannot pull flue gases upward effectively.
Possible causes: soot blockage, physical obstruction at top, structural damage to liner, downdraught from wind.
Action: Shut down boiler. Inspect chimney before restarting.

Draft between minus 2 and minus 5 Pascals (normal):

Good draft. Chimney is working correctly.
No action needed.

Draft between minus 5 and minus 10 Pascals (excess draft):

More suction than needed.
Excess cold air is being pulled through the system, reducing efficiency and lowering combustion temperature.
Possible causes: chimney too tall, flue gas temperature very high, strong wind conditions, air register too open.
Action: Consider installing a draft diverter or adjusting air register.

Draft above 0 Pascals (positive, back pressure):

Flue gases are flowing back into the boiler room. EVACUATE.
This is an emergency condition.
Combustion products including CO are entering the workspace.

RELATIONSHIP WITH FLUE TEMPERATURE:

Draft and flue temperature are closely related.
Hotter flue gases are less dense than cooler air outside.
This density difference creates the buoyancy force driving draft.
Higher flue temperature = stronger natural draft.
If flue temperature drops significantly, draft will also weaken.
In very hot summer weather, the temperature difference between flue gas and outside air is smaller, so draft may be 0.5 Pa weaker.
This is normal seasonal variation.

SEASONAL AND WEATHER EFFECTS:

Winter: Outside air is colder and denser.

Temperature difference between flue gas and outside air is larger.

Draft is naturally stronger in winter.

Draft may be 1 to 2 Pascals stronger in winter than in summer.

Strong wind: Can create downdraught (wind pushing air back down chimney)
or can enhance draft (Venturi effect at chimney top).

Effect depends on wind direction relative to chimney orientation.

TREND MONITORING:

If draft pressure becomes less negative by 0.5 Pa per month:

Soot buildup is accumulating, narrowing the chimney.

Schedule sweeping before draft falls below minus 2 Pa.

If draft pressure suddenly changes by more than 1 Pa:

Physical obstruction or structural event.

Inspect chimney before next operation.

"""

},

]

— Validation helper —————

```
def validate_documents():
```

```
    """Check all documents have required fields and unique IDs."""
```

```
    ids = []
```

```
    errors = []
```

```
    for i, doc in enumerate(KNOWLEDGE_DOCUMENTS):
```

```
        for field in ["id", "category", "title", "content"]:
```

```
            if field not in doc:
```

```
                errors.append(f"Document {i}: missing field '{field}'")
```

```
        if "id" in doc:
```

```
            if doc["id"] in ids:
```

```
                errors.append(f"Duplicate ID: '{doc['id']}'")
```

```
            ids.append(doc["id"])
```

```
        if "content" in doc and len(doc["content"].strip()) < 100:
```

```
            errors.append(f"Document '{doc.get('id')}': content too short (under
```

```
100 chars)")
```

```
    return errors
```

```
if __name__ == "__main__":
```

```
    errors = validate_documents()
```

```
    if errors:
```

```
        print("▣ Validation errors:")
```

```
        for e in errors:
```

```
            print(f"    {e}")
```

```
    else:
```

```
        print(f"▣ All {len(KNOWLEDGE_DOCUMENTS)} documents valid")
```

```
        categories = {}
```

```
        for doc in KNOWLEDGE_DOCUMENTS:
```

```
            cat = doc.get("category", "unknown")
```

```
            categories[cat] = categories.get(cat, 0) + 1
```

```
        for cat, count in categories.items():
```

```
            print(f"    {cat}: {count} documents")
```

PART 3 — INDEXER CODE

3.1 indexer.py — full code explained line by line

Create knowledge_base/indexer.py:

```
"""
knowledge_base/indexer.py

Indexes boiler_guides.py documents into ChromaDB.
Supports four modes:
    --mode=full      Delete everything and rebuild from scratch (safest)
    --mode=add        Add only new documents (by ID, skips existing)
    --mode=update     Update one specific document by ID
    --mode=verify     Check what is currently in ChromaDB without changing it

Usage:
    python indexer.py                (defaults to full re-index)
    python indexer.py --mode=full    (delete all and rebuild)
    python indexer.py --mode=add      (add documents not yet in DB)
    python indexer.py --mode=update --id=fault_high_co (update one doc)
    python indexer.py --mode=verify   (show what is in ChromaDB)
"""

import argparse
import sys
import os
sys.path.append("..")

import chromadb
from chromadb.utils import embedding_functions

# Import all documents
from boiler_guides import KNOWLEDGE_DOCUMENTS, validate_documents

# Import config
from config import CHROMA_PATH, CHROMA_COLLECTION, EMBEDDING_MODEL

# — Load embedding model —————
# This model converts text to vectors.
# It is downloaded once (~22MB) and cached locally.
# Always use the same model for indexing and querying.
print(f"▣ Loading embedding model: {EMBEDDING_MODEL}")
embedding_fn = embedding_functions.SentenceTransformerEmbeddingFunction(
    model_name=EMBEDDING_MODEL
)

# — Connect to ChromaDB —————
# PersistentClient means the database is saved to disk.
# The chroma_db/ folder is created automatically if it does not exist.
print(f"▣ Connecting to ChromaDB at: {CHROMA_PATH}")
chroma_client = chromadb.PersistentClient(path=CHROMA_PATH)

def get_or_create_collection():
    """Get existing collection or create new one."""
    try:
        collection = chroma_client.get_collection(
            name=CHROMA_COLLECTION,
            embedding_function=embedding_fn,
        )
```

```

        print(f"▢ Found existing collection: {CHROMA_COLLECTION}
({collection.count()} docs)")
        return collection
    except Exception:
        collection = chroma_client.create_collection(
            name=CHROMA_COLLECTION,
            embedding_function=embedding_fn,
            metadata={"hnsw:space": "cosine"},
            # hnsw:space=cosine means similarity is measured by cosine distance
            # cosine is better than euclidean for text similarity
        )
        print(f"▢ Created new collection: {CHROMA_COLLECTION}")
        return collection

def mode_full():
    """
    DELETE everything and rebuild from scratch.
    Use this when:
    - You made major changes to multiple documents
    - You deleted documents from boiler_guides.py
    - Something seems wrong with the existing index
    - You want a guaranteed clean state
    """
    print("\n▢ MODE: FULL RE-INDEX")
    print("  This will DELETE all existing documents and rebuild.")

    # Validate documents first
    errors = validate_documents()
    if errors:
        print("▢ Validation failed. Fix errors before indexing:")
        for e in errors:
            print(f"    {e}")
        sys.exit(1)

    # Delete existing collection
    try:
        chroma_client.delete_collection(CHROMA_COLLECTION)
        print("  🗑 Deleted existing collection")
    except Exception:
        print("  i No existing collection to delete")

    # Create fresh collection
    collection = chroma_client.create_collection(
        name=CHROMA_COLLECTION,
        embedding_function=embedding_fn,
        metadata={"hnsw:space": "cosine"},
    )

    # Index all documents
    docs, ids, metas = [], [], []
    for doc in KNOWLEDGE_DOCUMENTS:
        docs.append(doc["content"].strip())
        ids.append(doc["id"])
        metas.append({
            "title": doc["title"],
            "category": doc.get("category", "general"),
        })

    # collection.add() calls embedding_fn on each document automatically
    # It stores: original text + vector + metadata, all linked by ID
    collection.add(documents=docs, ids=ids, metadatas=metas)

    print(f"\n▢ FULL RE-INDEX COMPLETE")

```

```

print(f"    Total documents indexed: {collection.count()}")
for doc in KNOWLEDGE_DOCUMENTS:
    print(f"    ✓ {doc['id']} – {doc['title']}")

def mode_add():
    """
    Add only documents that do not already exist in ChromaDB.
    Use this when:
    - You added new documents to boiler_guides.py
    - You do not want to disturb existing indexed documents
    Safe to run multiple times – skips documents already indexed.
    """
    print("\n▢ MODE: ADD NEW DOCUMENTS ONLY")

    collection = get_or_create_collection()

    # Find which document IDs are already in ChromaDB
    existing_ids = set()
    try:
        existing = collection.get(include=[]) # get IDs only, no content
        existing_ids = set(existing["ids"])
        print(f"    Found {len(existing_ids)} existing documents in ChromaDB")
    except Exception as e:
        print(f"    Could not fetch existing IDs: {e}")

    # Filter to only new documents
    new_docs = [d for d in KNOWLEDGE_DOCUMENTS if d["id"] not in existing_ids]
    skip_docs = [d for d in KNOWLEDGE_DOCUMENTS if d["id"] in existing_ids]

    if not new_docs:
        print(f"    ▢ Nothing to add. All {len(KNOWLEDGE_DOCUMENTS)} documents
already indexed.")
        return

    print(f"    Skipping {len(skip_docs)} existing documents")
    print(f"    Adding {len(new_docs)} new documents")

    docs, ids, metas = [], [], []
    for doc in new_docs:
        docs.append(doc["content"].strip())
        ids.append(doc["id"])
        metas.append({
            "title": doc["title"],
            "category": doc.get("category", "general"),
        })

    collection.add(documents=docs, ids=ids, metadatas=metas)

    print(f"\n▢ ADD COMPLETE")
    for doc in new_docs:
        print(f"    ✓ Added: {doc['id']} – {doc['title']}")
    print(f"    Total in collection: {collection.count()}")

def mode_update(doc_id: str):
    """
    Update one specific document by its ID.
    Use this when:
    - You edited the content of one document in boiler_guides.py
    - You want only that document's vector to be recalculated
    Does not affect any other documents.
    """
    print(f"\n▢ MODE: UPDATE DOCUMENT – {doc_id}")

```

```

# Find the document in boiler_guides.py
doc = next((d for d in KNOWLEDGE_DOCUMENTS if d["id"] == doc_id), None)
if doc is None:
    print(f" Document ID '{doc_id}' not found in boiler_guides.py")
    print(f" Available IDs: {[d['id'] for d in KNOWLEDGE_DOCUMENTS]}")
    sys.exit(1)

```

```

collection = get_or_create_collection()

```

```

# Check if document exists in ChromaDB

```

```

try:
    existing = collection.get(ids=[doc_id])
    if existing["ids"]:
        # Document exists – update it
        # collection.update() recalculates the vector for new content
        collection.update(
            ids=[doc_id],
            documents=[doc["content"].strip()],
            metadatas=[{
                "title": doc["title"],
                "category": doc.get("category", "general"),
            }],
        )
        print(f" Updated: {doc_id}")
        print(f" Title: {doc['title']}")
        print(f" Content length: {len(doc['content'])} characters")
    else:
        # Document does not exist – add it
        collection.add(
            ids=[doc_id],
            documents=[doc["content"].strip()],
            metadatas=[{
                "title": doc["title"],
                "category": doc.get("category", "general"),
            }],
        )
        print(f" ADDED (was not in ChromaDB): {doc_id}")
except Exception as e:
    print(f" Error updating {doc_id}: {e}")
    sys.exit(1)

```

```

def mode_verify():

```

```

    """
    Show what is currently stored in ChromaDB.
    Does not modify anything.
    Use this to check the state before or after any operation.
    """
    print("\n🔍 MODE: VERIFY – showing current ChromaDB contents")

```

```

    try:
        collection = chroma_client.get_collection(
            name=CHROMA_COLLECTION,
            embedding_function=embedding_fn,
        )
    except Exception:
        print(" Collection not found. Run: python indexer.py --mode=full")
        return

```

```

    total = collection.count()
    print(f"\nCollection: {CHROMA_COLLECTION}")
    print(f"Total documents: {total}")

```

```

    if total == 0:
        print("⚠ Collection exists but is empty. Run: python indexer.py --mode=full")
        return

    # Get all document IDs and metadata
    all_docs = collection.get(include=["metadatas", "documents"])

    print(f"\nIndexed documents:")
    for doc_id, meta, doc_text in zip(
        all_docs["ids"],
        all_docs["metadatas"],
        all_docs["documents"]
    ):
        in_guides = any(d["id"] == doc_id for d in KNOWLEDGE_DOCUMENTS)
        status = "✓" if in_guides else "⚠ NOT IN boiler_guides.py (orphan)"
        print(f" {status} {doc_id}")
        print(f"      Title: {meta.get('title', 'N/A')}")
        print(f"      Category: {meta.get('category', 'N/A')}")
        print(f"      Content: {len(doc_text)} characters")

    # Check for documents in boiler_guides.py but NOT in ChromaDB
    indexed_ids = set(all_docs["ids"])
    guide_ids = set(d["id"] for d in KNOWLEDGE_DOCUMENTS)
    missing = guide_ids - indexed_ids
    if missing:
        print(f"\n⚠ In boiler_guides.py but NOT indexed:")
        for mid in missing:
            print(f"  x {mid}")
        print(f"  Run: python indexer.py --mode=add to index these")

    # Test a sample query
    print(f"\nQ Test query: 'boiler pressure high fault'")
    results = collection.query(
        query_texts=["boiler pressure high fault"],
        n_results=3,
    )
    print("  Top 3 results:")
    for meta, dist in zip(results["metadatas"][0], results["distances"][0]):
        similarity = round((1 - dist) * 100, 1)
        print(f"    → {meta['title']} (similarity: {similarity}%)")

# — CLI entry point —
if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Knowledge Base Indexer")
    parser.add_argument(
        "--mode",
        choices=["full", "add", "update", "verify"],
        default="full",
        help="Operation mode"
    )
    parser.add_argument(
        "--id",
        default=None,
        help="Document ID for update mode"
    )
    args = parser.parse_args()

    if args.mode == "full":
        mode_full()
    elif args.mode == "add":
        mode_add()
    elif args.mode == "update":

```

```
        if not args.id:
            print("▯ --id is required for update mode")
            print("    Example: python indexer.py --mode=update --
id=fault_high_co")
            sys.exit(1)
        mode_update(args.id)
    elif args.mode == "verify":
        mode_verify()
```

3.2 How to run it

```
# First time – build everything from scratch
cd knowledge_base
pip install chromadb sentence-transformers
python indexer.py --mode=full
```

```
# Expected output:
# ▯ Loading embedding model: all-MiniLM-L6-v2
# ▯ Connecting to ChromaDB at: ./chroma_db
# ▯ MODE: FULL RE-INDEX
# 🗑 Deleted existing collection
# ▯ FULL RE-INDEX COMPLETE
#   Total documents indexed: 12
#   ✓ fault_high_pressure – HIGH_PRESSURE Fault – Complete Guide
#   ✓ fault_low_water_level – LOW_WATER_LEVEL Fault – Complete Guide
#   ... (all 12 listed)
```

3.3 How to verify it worked

```
python indexer.py --mode=verify
```

```
# Expected output:
# 🔍 MODE: VERIFY – showing current ChromaDB contents
#
# Collection: boiler_knowledge
# Total documents: 12
#
# Indexed documents:
#   ✓ fault_high_pressure
#       Title: HIGH_PRESSURE Fault – Complete Guide
#       Category: boiler_fault
#       Content: 2847 characters
#   ✓ fault_low_water_level
#       Title: LOW_WATER_LEVEL Fault – Complete Guide
#   ... (all 12 shown)
#
# 🔍 Test query: 'boiler pressure high fault'
#   Top 3 results:
#   → HIGH_PRESSURE Fault Guide (similarity: 94.2%)
#   → Combined Diagnosis: HIGH_PRESSURE + LOW_WATER (similarity: 78.1%)
#   → HIGH_TEMPERATURE Fault Guide (similarity: 61.3%)
```

PART 4 — KNOWLEDGE BASE CHANGES

The 4 scenarios when knowledge base changes

SCENARIO 1: You edited an existing document

Example: You improved the HIGH_CO guide with better action steps.
What changed: The content text of one document in boiler_guides.py.
Problem: ChromaDB still has the OLD vector for that document.
The old vector points to old meaning.
Queries will retrieve the old content.
Solution: Update only that document's vector.

Command: `python indexer.py --mode=update --id=fault_high_co`
Time: ~5 seconds
Affects: Only that one document

SCENARIO 2: You added a new document

Example: You added a new "LOW_FUEL_PRESSURE" fault guide.
What changed: A new dict added to KNOWLEDGE_DOCUMENTS list.
Problem: ChromaDB does not know this document exists.
Queries will never retrieve it.
Solution: Add only the new document without touching existing ones.

Command: `python indexer.py --mode=add`
Time: ~5 seconds
Affects: Only adds documents not already in ChromaDB

SCENARIO 3: You deleted a document

Example: You removed the ABNORMAL_AIRFLOW guide (decided it was redundant).
What changed: You deleted that dict from KNOWLEDGE_DOCUMENTS.
Problem: ChromaDB still has the old document with its vector.
Queries may still retrieve it even though you deleted it.
Solution: Full re-index to get a clean state.

Command: `python indexer.py --mode=full`
Time: ~30 seconds
Affects: Deletes everything and rebuilds from current boiler_guides.py

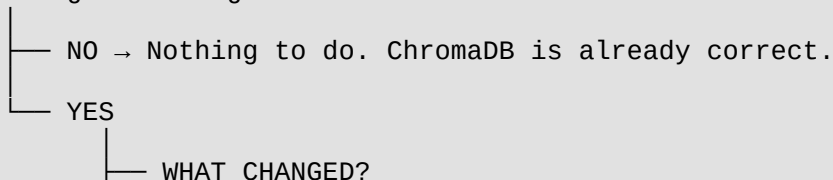
SCENARIO 4: Major restructure (multiple edits + adds + deletes)

Example: You rewrote 5 documents and added 3 new ones.
Solution: Full re-index is simplest and safest.

Command: `python indexer.py --mode=full`
Time: ~30 seconds

The decision tree — which command to use

Did you change knowledge base?



- Edited content of 1 document
 - └─ `python indexer.py --mode=update --id=<the_id>`
- Added 1 or more new documents
 - └─ `python indexer.py --mode=add`
- Deleted 1 or more documents
 - └─ `python indexer.py --mode=full`
- Multiple changes (edits + adds + deletes mixed)
 - └─ `python indexer.py --mode=full`

Safe update workflow — always follow this order

- Step 1: STOP the FastAPI server
(if server is running, it holds ChromaDB collection open)
Ctrl+C in the terminal running uvicorn
- Step 2: Make your changes to `boiler_guides.py`
- Step 3: Validate your changes
`python boiler_guides.py`
(runs the `validate_documents()` function at bottom of file)
- Step 4: Run the appropriate indexer command
- Step 5: Verify the result
`python indexer.py --mode=verify`
- Step 6: RESTART the FastAPI server
`uvicorn api.chatbot_api:app --host 0.0.0.0 --port 8000 --reload`
- Step 7: Test a query related to your change
`curl -X POST http://localhost:8000/chat \`
`-H "Content-Type: application/json" \`
`-d '{"question": "test query related to your changed document"}'`

Production version strategy

When you deliver to a real client, track your knowledge base changes like code:

```
knowledge_base/
├─ boiler_guides.py           ← current version
├─ boiler_guides_v1.py       ← original version (backup)
├─ boiler_guides_v2.py       ← second version (backup)
└─ CHANGELOG.md              ← what changed and when
```

CHANGELOG.md example:

```
## v3 — 2024-02-15
- UPDATED fault_high_co: Added CO ppm safety thresholds table
- ADDED fault_low_fuel_flow: New document for fuel flow fault
- Command used: python indexer.py --mode=add

## v2 — 2024-02-01
- UPDATED fault_blocked_flue: Added downdraught causes
- Command used: python indexer.py --mode=update --id=fault_blocked_flue

## v1 — 2024-01-20
- Initial 12 documents indexed
```

```
- Command used: python indexer.py --mode=full
```

PART 5 — RAGAS EVALUATION CONCEPTS

5.1 What RAGAS is and why you need it

RAGAS stands for Retrieval Augmented Generation Assessment. It is a Python library that automatically measures how good your RAG chatbot's answers are.

Without evaluation you are guessing:

```
You: "Does the chatbot work well?"
You: "Uh... the answers look okay to me?"
Client: "How accurate is it? Give me a number."
You: "..."
```

With RAGAS evaluation you can say:

```
You: "The chatbot achieves 0.91 faithfulness, 0.88 answer relevancy,
      0.92 tool precision, with average response time of 3.2 seconds.
      These scores are logged to Grafana and visible in real time."
Client: "Perfect."
```

RAGAS measures three things after every single chatbot answer.

5.2 The 3 metrics explained with boiler examples

Metric 1: Faithfulness (0 to 1)

Question: Did the answer ONLY use information from the retrieved context?

This measures whether the chatbot is hallucinating (making things up) or staying grounded in the actual data that was retrieved.

EXAMPLE OF HIGH FAITHFULNESS (score near 1.0):

```
Context retrieved: "HIGH_PRESSURE fault: reduce burner rate,
                  check relief valve, open steam outlet"
Answer given:      "Reduce burner firing rate immediately.
                  Check the pressure relief valve operation.
                  Open the main steam outlet valve."
Faithfulness: 0.95 ← answer used only what context said ◻
```

EXAMPLE OF LOW FAITHFULNESS (score near 0.0):

```
Context retrieved: "HIGH_PRESSURE fault: reduce burner rate,
                  check relief valve, open steam outlet"
Answer given:      "Check the expansion vessel, bleed the
                  radiators, call your gas network provider,
                  and check the motorised zone valves."
Faithfulness: 0.12 ← answer invented things not in context ◻
```

The model used domestic heating knowledge
not present in the industrial boiler context

Low faithfulness means: your fine-tuned model is ignoring the retrieved context and answering from its training memory instead. Fix: make the system instruction stronger — "Only use tool results to answer."

Metric 2: Answer Relevancy (0 to 1)

Question: Does the answer actually address what the user asked?

EXAMPLE OF HIGH RELEVANCY (score near 1.0):

Question: "Why is CO high in the chimney?"
Answer: "CO is high because of insufficient combustion air reaching the burner. The O2 reading of 2.1% confirms this. Increase air damper by 10%. Check air filter."
Relevancy: 0.93 ← directly answers the WHY question ▢

EXAMPLE OF LOW RELEVANCY (score near 0.0):

Question: "Why is CO high in the chimney?"
Answer: "The boiler pressure is currently 12.3 bar which is within the normal range of 10 to 14 bar. Water level is 52% which is also normal. Temperature is 87°C."
Relevancy: 0.08 ← answered a different question entirely ▢
Gave sensor readings when asked about CO cause

Low relevancy means: the wrong knowledge base documents were retrieved, OR the tool schemas are not clear enough for the agent to call the right tools. Fix: improve tool schema descriptions and add more specific knowledge documents.

Metric 3: Tool Precision (0 to 1) — custom metric

Question: Did the agent call the tools it SHOULD have called for this type of question?

This is a custom metric not in base RAGAS. It checks agent decision-making quality.

QUESTION TYPE → EXPECTED TOOLS

"Is boiler safe now?"
→ Should call: fetch_realtime_sensors, get_fault_history
→ Called: fetch_realtime_sensors, get_fault_history
→ Precision: 2/2 = 1.0 ▢

"Why is pressure high?"
→ Should call: fetch_realtime_sensors, search_knowledge_base, get_fault_history
→ Called: fetch_realtime_sensors, search_knowledge_base
→ Precision: 2/3 = 0.67 ▲ (missed get_fault_history)

"Will pressure reach critical?"
→ Should call: fetch_realtime_sensors, predict_trend
→ Called: fetch_realtime_sensors
→ Precision: 1/2 = 0.5 ▢ (missed predict_trend)

Low tool precision means: tool schema descriptions need improvement. The agent is not understanding when to use which tool.

5.3 What scores mean and how to act on them

FAITHFULNESS

Score	Status	What to do
> 0.85	▣ Good	No action needed
0.70-0.85	⚠ OK	Review recent answers. Strengthen system prompt.
< 0.70	▣ Bad	Agent is hallucinating. Critical fix needed. Add to system: "NEVER use knowledge not from tools."

ANSWER RELEVANCY

Score	Status	What to do
> 0.85	▣ Good	No action needed
0.70-0.85	⚠ OK	Check which queries score low. Add more specific docs.
< 0.70	▣ Bad	Wrong documents being retrieved. Rewrite knowledge base document titles and content to use more specific terminology.

TOOL PRECISION

Score	Status	What to do
> 0.88	▣ Good	No action needed
0.70-0.88	⚠ OK	Check which tool is being missed. Improve that tool's schema description.
< 0.70	▣ Bad	Agent making poor tool decisions. Review all tool schemas. Make descriptions more specific.

LATENCY (milliseconds)

< 3000ms	▣ Fast	Excellent user experience
3000-6000	⚠ OK	Acceptable
6000-9000	▣ Slow	Investigate. Reduce MAX_AGENT_STEPS or tool response time.
> 9000ms	▣ Bad	Check Vertex AI quota limits. Check InfluxDB query speed.

PART 6 — RAGAS EVALUATOR CODE

6.1 evaluator.py — full code explained line by line

Create evaluation/evaluator.py:

```
"""
evaluation/evaluator.py

Measures quality of every Agentic RAG answer using:
1. RAGAS faithfulness — no hallucination
2. RAGAS answer_relevancy — answers the actual question
3. Custom tool_precision — called the right tools
4. Latency measurement — response speed
5. Steps tracking — how many tool calls were needed

All scores are stored in InfluxDB automatically.
```

```

View scores in Grafana using the chatbot_evaluation measurement.
"""

import sys, os
sys.path.append("../")

from ragas import evaluate
from ragas.metrics import faithfulness, answer_relevancy
from datasets import Dataset

from langchain_google_vertexai import ChatVertexAI
from langchain_community.embeddings import HuggingFaceEmbeddings

from influxdb_client import InfluxDBClient, Point
from influxdb_client.client.write_api import SYNCHRONOUS

from datetime import datetime
import numpy as np

from config import (
    INFLUX_URL, INFLUX_TOKEN, INFLUX_ORG, INFLUX_BUCKET,
    GCP_PROJECT, GCP_LOCATION, EMBEDDING_MODEL,
)

# — InfluxDB write client —————
# Created once, reused for all score logging
_influx_client = InfluxDBClient(url=INFLUX_URL, token=INFLUX_TOKEN,
org=INFLUX_ORG)
_write_api = _influx_client.write_api(write_options=SYNCHRONOUS)

# — Expected tools per question type (for tool_precision metric) —————
# Maps keywords found in questions to tools that SHOULD be called.
# Used to calculate tool_precision score.
EXPECTED_TOOLS_MAP = {
    "fetch_realtime_sensors": [
        "current", "now", "right now", "safe", "reading", "value",
        "temperature", "pressure", "status", "level", "is it",
        "how much", "what is the",
    ],
    "search_knowledge_base": [
        "why", "cause", "reason", "explain", "what is", "how to fix",
        "how do i", "what does", "fault", "guide", "what happens",
        "dangerous", "mean", "definition",
    ],
    "get_fault_history": [
        "recent", "history", "occurred", "happened", "last hour",
        "last 24", "yesterday", "fault log", "events", "before",
        "when did", "how many times",
    ],
    "predict_trend": [
        "predict", "trend", "will it", "going to", "rising",
        "falling", "how long", "when will", "future", "increase",
        "get worse", "before it reaches",
    ],
}

def get_expected_tools(question: str) -> list:
    """
    Determine which tools SHOULD have been called based on question keywords.
    Returns a list of expected tool names.
    """

```

```

question_lower = question.lower()
expected = []
for tool, keywords in EXPECTED_TOOLS_MAP.items():
    if any(kw in question_lower for kw in keywords):
        expected.append(tool)
return expected if expected else ["fetch_realtime_sensors"]

```

```

def calculate_tool_precision(tools_actually_called: list, question: str) ->
float:

```

```

    """
    Calculates what fraction of expected tools were actually called.

    Formula: tools correctly called / total expected tools

    Examples:
    Expected: [fetch_realtime, search_knowledge]
    Called:   [fetch_realtime, search_knowledge]
    Score:    2/2 = 1.0

    Expected: [fetch_realtime, search_knowledge, get_fault_history]
    Called:   [fetch_realtime, search_knowledge]
    Score:    2/3 = 0.67

    Expected: [predict_trend, fetch_realtime]
    Called:   [fetch_realtime]
    Score:    1/2 = 0.5
    """

```

```

    expected = get_expected_tools(question)
    if not expected:
        return 1.0

    correctly_called = set(tools_actually_called) & set(expected)
    precision = len(correctly_called) / len(expected)
    return round(precision, 3)

```

```

class BoilerEvaluator:

```

```

    def __init__(self):
        """
        Initialise the evaluation engine.
        Uses base Gemini (NOT your fine-tuned model) as the judge.
        The judge model evaluates quality – it should be general, not domain-
specific.
        """
        print("▯ Loading evaluation engine...")

        # Judge LLM – base Gemini for evaluation
        # We use the base model because RAGAS asks it meta-questions like:
        # "Does this answer use only information from this context?"
        # A domain-specific model might be biased in these judgements.
        self.judge_llm = ChatVertexAI(
            model_name="gemini-1.5-flash",    # base model, not fine-tuned
            project=GCP_PROJECT,
            location=GCP_LOCATION,
            temperature=0,    # deterministic for consistent evaluation
        )

        # Embedding model for answer_relevancy calculation
        # RAGAS uses embeddings to compare question and answer semantically
        self.embeddings = HuggingFaceEmbeddings(
            model_name=EMBEDDING_MODEL
        )

```

```

        print("\n Evaluation engine ready")

def evaluate_answer(
    self,
    question:          str,      # the user's original question
    answer:             str,      # the chatbot's response
    contexts:           list,     # list of strings – tool results used by
agent
    latency_ms:         float,    # how long the response took in milliseconds
    steps_taken:        int,      # how many tool calls the agent made
    tools_actually_called: list,   # list of tool names that were called
) -> dict:
    """
    Run all evaluation metrics on one question-answer pair.

    This function is called ONCE after every /chat API request.
    It runs 3 metrics, calculates scores, logs to InfluxDB, and returns
scores.

    Args:
        question: The user's original question text
        answer: The final answer text from the agent
        contexts: List of strings – each string is a tool result
                   (sensor data, knowledge base docs, fault history, etc.)
        latency_ms: Total time from question to answer in milliseconds
        steps_taken: Total number of tool calls made in this interaction
        tools_actually_called: Names of tools called (e.g.
["fetch_realtime_sensors", "search_knowledge_base"])

    Returns:
        Dict with all scores including faithfulness, answer_relevancy,
        tool_precision, overall_quality, and latency_ms
    """

    print(f"\n E Evaluating answer for: '{question[:60]}...'")

    # — Metric 1 + 2: RAGAS Faithfulness + Answer Relevancy —————
    # RAGAS needs a specific format: Dataset with question, answer, contexts
    # contexts must be a list of strings (not a single string)
    eval_dataset = Dataset.from_dict({
        "question": [question],
        "answer":    [answer],
        "contexts": [contexts],    # list of strings, wrapped in outer list
    })

    faithfulness_score = 0.0
    relevancy_score    = 0.0

    try:
        result = evaluate(
            dataset=eval_dataset,
            metrics=[faithfulness, answer_relevancy],
            llm=self.judge_llm,
            embeddings=self.embeddings,
            raise_exceptions=False,    # don't crash on individual metric
failures
        )

        df = result.to_pandas()

        # Extract scores safely (they could be NaN if evaluation failed)
        raw_faith = df["faithfulness"].iloc[0]
        raw_relev = df["answer_relevancy"].iloc[0]

```

```

        faithfulness_score = round(float(raw_faith) if raw_faith ==
raw_faith else 0.0, 3)
        relevancy_score = round(float(raw_relev) if raw_relev ==
raw_relev else 0.0, 3)

        print(f"    faithfulness:    {faithfulness_score}")
        print(f"    answer_relevancy: {relevancy_score}")

    except Exception as e:
        print(f"    ⚠ RAGAS evaluation error: {e}")
        print(f"    Using default scores (0.0) for this query")

    # — Metric 3: Tool Precision (custom) —————
    tool_precision = calculate_tool_precision(tools_actually_called,
question)
    print(f"    tool_precision:    {tool_precision} "
          f"(called: {tools_actually_called}, "
          f"expected: {get_expected_tools(question)})")

    # — Overall Quality: weighted average —————
    # Faithfulness and relevancy are most important (0.4 each)
    # Tool precision is also important (0.2)
    overall_quality = round(
        (faithfulness_score * 0.4) +
        (relevancy_score * 0.4) +
        (tool_precision * 0.2),
        3
    )
    print(f"    overall_quality: {overall_quality}")
    print(f"    latency_ms:          {latency_ms}")
    print(f"    steps_taken:         {steps_taken}")

    # — Package all scores —————
    scores = {
        "faithfulness":    faithfulness_score,
        "answer_relevancy": relevancy_score,
        "tool_precision":  tool_precision,
        "overall_quality": overall_quality,
        "latency_ms":      latency_ms,
        "steps_taken":     steps_taken,
        "tools_used":      ", ".join(tools_actually_called),
        "timestamp":       datetime.utcnow().isoformat() + "Z",
    }

    # — Log scores to InfluxDB —————
    self._log_to_influx(question, scores)

    return scores

def _log_to_influx(self, question: str, scores: dict):
    """
    Write evaluation scores to InfluxDB measurement: chatbot_evaluation
    These are automatically visible in Grafana.
    """
    try:
        point = (
            Point("chatbot_evaluation")
            # Tags (indexed, used for filtering in Grafana)
            .tag("question_preview", question[:60])

            # Fields (the actual numbers)
            .field("faithfulness", scores["faithfulness"])
            .field("answer_relevancy", scores["answer_relevancy"])

```

```

        .field("tool_precision",    scores["tool_precision"])
        .field("overall_quality",   scores["overall_quality"])
        .field("latency_ms",        scores["latency_ms"])
        .field("steps_taken",       float(scores["steps_taken"]))
        .field("tools_used",        scores["tools_used"])

    .time(scores["timestamp"])
)

_write_api.write(bucket=INFLUX_BUCKET, org=INFLUX_ORG, record=point)
print(f"    ▫ Scores logged to InfluxDB")

except Exception as e:
    # Evaluation logging failure should never crash the API
    print(f"    ⚠ Failed to log to InfluxDB: {e}")
    print(f"    Scores: {scores}")

```

6.2 How to integrate evaluator into chatbot_api.py

In your `api/chatbot_api.py`, find the `/chat` endpoint and update it:

```

@app.post("/chat")
def chat(request: ChatRequest):

    # Step 1: Run the agent (get answer + steps + tools called)
    result = agent.run(request.question)

    # Step 2: Collect contexts for RAGAS
    # contexts = all tool results combined as list of strings
    # RAGAS reads these to judge whether the answer used them faithfully
    contexts = []
    tools_called = []
    for step in result["steps"]:
        contexts.append(
            f"Tool: {step['tool']}\nResult: {step['result_preview']}"
        )
        tools_called.append(step["tool"])

    # Step 3: Evaluate (if contexts exist and evaluate=True)
    eval_scores = {}
    if request.evaluate and contexts:
        eval_scores = evaluator.evaluate_answer(
            question=request.question,
            answer=result["answer"],
            contexts=contexts,
            latency_ms=result["latency_ms"],
            steps_taken=result["total_steps"],
            tools_actually_called=tools_called,
        )

    return {
        "question":    request.question,
        "answer":      result["answer"],
        "steps":       result["steps"],
        "total_steps": result["total_steps"],
        "latency_ms":  result["latency_ms"],
        "eval_scores": eval_scores,
        "timestamp":   result["timestamp"],
    }

```

6.3 How to see scores in Grafana

Since your Grafana is already running and connected to InfluxDB, create a new dashboard called "Chatbot Evaluation".

Panel 1 — Overall Quality (Gauge)

```
from(bucket: "boiler_data")
  |> range(start: -24h)
  |> filter(fn: (r) => r["_measurement"] == "chatbot_evaluation")
  |> filter(fn: (r) => r["_field"] == "overall_quality")
  |> mean()
```

Type: **Gauge** | Min: 0 | Max: 1 Thresholds:

- 0.0 to 0.5 → Red (poor)
- 0.5 to 0.75 → Yellow (acceptable)
- 0.75 to 1.0 → Green (good)

Panel 2 — All 3 Scores Over Time (Time Series)

```
from(bucket: "boiler_data")
  |> range(start: -24h)
  |> filter(fn: (r) => r["_measurement"] == "chatbot_evaluation")
  |> filter(fn: (r) =>
    r["_field"] == "faithfulness" or
    r["_field"] == "answer_relevancy" or
    r["_field"] == "tool_precision")
```

Type: **Time series** Shows 3 lines. You can see if a particular metric drops after a change.

Panel 3 — Response Latency (Time Series)

```
from(bucket: "boiler_data")
  |> range(start: -24h)
  |> filter(fn: (r) => r["_measurement"] == "chatbot_evaluation")
  |> filter(fn: (r) => r["_field"] == "latency_ms")
```

Type: **Time series** | Unit: **milliseconds (ms)**

Panel 4 — Average Steps Per Query (Stat)

```
from(bucket: "boiler_data")
  |> range(start: -24h)
  |> filter(fn: (r) => r["_measurement"] == "chatbot_evaluation")
  |> filter(fn: (r) => r["_field"] == "steps_taken")
  |> mean()
```

Type: **Stat** Tells you: on average, how many tools does the agent call per question? Target: 2 to 3 steps per question. Above 4 steps consistently = agent is over-thinking. Review tool schemas.

Panel 5 — 24h Score Summary (4 Stat panels side by side)

One stat panel each for:

```
# faithfulness 24h average
from(bucket: "boiler_data")
```

```
|> range(start: -24h)
|> filter(fn: (r) => r["_measurement"] == "chatbot_evaluation")
|> filter(fn: (r) => r["_field"] == "faithfulness")
|> mean()
```

Repeat for answer_relevancy, tool_precision, latency_ms.

PART 7 — COMPLETE STARTUP SEQUENCE

Run these in order. Every step matters.

```
# — STEP 1: Infrastructure (already running from before) —————
docker-compose up -d emqx influxdb grafana

# — STEP 2: Validate knowledge base documents —————
cd knowledge_base
python boiler_guides.py
# Expected: □ All 12 documents valid

# — STEP 3: Build ChromaDB knowledge base —————
python indexer.py --mode=full
# Expected: □ FULL RE-INDEX COMPLETE – 12 documents

# — STEP 4: Verify knowledge base —————
python indexer.py --mode=verify
# Expected: All 12 documents listed + test query returns HIGH_PRESSURE first
cd ..

# — STEP 5: Install evaluation dependencies —————
pip install ragas langchain-google-vertexai langchain-community

# — STEP 6: Start simulators (already set up) —————
python simulators/boiler_simulator.py &
python simulators/chimney_simulator.py &

# — STEP 7: Start consumers (already set up) —————
python consumers/influx_consumer.py &
python consumers/fault_detector.py &

# — STEP 8: Start chatbot API —————
uvicorn api.chatbot_api:app --host 0.0.0.0 --port 8000 --reload

# — STEP 9: Test the full system —————
curl -X POST http://localhost:8000/chat \
  -H "Content-Type: application/json" \
  -d '{"question": "Why is CO high in the chimney and how do I fix it?",
    "evaluate": true}'

# Expected response includes:
# - answer: specific diagnosis using live data + knowledge base
# - steps: shows fetch_realtime_sensors + search_knowledge_base called
# - eval_scores: faithfulness ~0.90, relevancy ~0.88, tool_precision ~1.0
# - latency_ms: under 6000

# — STEP 10: Open Grafana —————
# Go to: http://localhost:3000
# Navigate to: Chatbot Evaluation dashboard
# You should see scores appearing in real time as you send queries
```

QUICK REFERENCE — KNOWLEDGE BASE COMMANDS

```
# First time setup
python knowledge_base/indexer.py --mode=full

# Add new documents to boiler_guides.py, then:
python knowledge_base/indexer.py --mode=add

# Edit one document in boiler_guides.py, then:
python knowledge_base/indexer.py --mode=update --id=the_document_id

# Delete documents from boiler_guides.py, then:
python knowledge_base/indexer.py --mode=full

# Check current state:
python knowledge_base/indexer.py --mode=verify

# Validate document format:
python knowledge_base/boiler_guides.py
```

QUICK REFERENCE — EVALUATION SCORES

Metric	Target	Low = Problem	Fix
faithfulness	> 0.85	Model hallucinating	Stronger system prompt
answer_relevancy	> 0.85	Wrong docs retrieved	Better knowledge docs
tool_precision	> 0.88	Wrong tools called	Better tool schemas
latency_ms	< 4000	Too slow	Reduce MAX_AGENT_STEPS
steps_taken avg	2-3	Too many steps (>4)	Reduce MAX_AGENT_STEPS
overall_quality	> 0.85	General quality poor	Review all of the above

Stack: ChromaDB · all-MiniLM-L6-v2 · RAGAS · Vertex AI · InfluxDB · Grafana